PULSO

ENTREGA 1 / E5

Arquitectura del sistema C4

Proyecto Final Backend · 13 de noviembre de 2026

Miguel Ángel Márquez Coronel · María Gabriela Rey Barrios · Andrés Felipe Serrano Mendoza

7. E5. Arquitectura del sistema mediante el modelo C4

7.1. Descripción general de la arquitectura

PULSO es una plataforma web educativa de prevención cardiovascular cuya arquitectura integra una interfaz de usuario, un backend de coordinación, un servicio independiente de Machine Learning y un sistema de generación de recomendaciones educativas mediante recuperación aumentada por generación (*Retrieval-Augmented Generation*, RAG).

La arquitectura está diseñada para separar las responsabilidades de presentación, coordinación de operaciones, inferencia predictiva y recuperación documental, facilitando el mantenimiento, las pruebas automatizadas y la evolución de los componentes.

El sistema utiliza las siguientes tecnologías:

Componente	Tecnología	Responsabilidad
Frontend	React, TypeScript y Vite	Captura de variables, presentación de resultados, explicaciones y simulaciones
Backend	Python y FastAPI	Casos de uso, autenticación JWT, persistencia y comunicación con servicios
Base relacional	PostgreSQL 16, SQLAlchemy y psycopg	Cuentas y resultados persistentes
ML Service	Python, FastAPI, XGBoost y SHAP	Predicción, explicabilidad y simulación
Sistema RAG	n8n	Orquestación de recomendaciones educativas
Base vectorial	Supabase, PostgreSQL y pgvector	Almacenamiento y recuperación del corpus documental
Proveedor de IA	Google Gemini API	Servicios de embeddings y generación de contenido
Proxy inverso	NGINX	Punto de entrada y enrutamiento HTTP
Contenerización	Docker y Docker Compose	Ejecución coordinada de los servicios principales

El backend está organizado siguiendo una adaptación de **Clean Architecture**, en la que las reglas del dominio y los casos de uso se mantienen desacoplados de las implementaciones tecnológicas.

Para documentar la solución se utiliza el modelo C4, que representa progresivamente el contexto general del sistema, sus contenedores de ejecución y los componentes internos del backend.

7.2. C4 Nivel 1 — Contexto del sistema

El diagrama de contexto presenta PULSO como una única unidad de software e identifica los actores que interactúan con la plataforma y sus dependencias externas.

7.2.1. Actores

Persona usuaria: puede registrarse, iniciar sesión, introducir factores cardiovasculares, consultar resultados, explicaciones SHAP, simulaciones, recomendaciones educativas y su historial de análisis.

Responsable técnico: administra y supervisa la infraestructura de PULSO, verifica el funcionamiento de los servicios y realiza operaciones de mantenimiento sobre el corpus documental del sistema RAG.

Este actor no representa necesariamente un usuario autenticado dentro de la aplicación, sino una responsabilidad técnica de operación y mantenimiento.

7.2.2. Sistema principal

PULSO: plataforma web educativa de orientación cardiovascular con autenticación de usuarios, predicción mediante Machine Learning, SHAP, simulación, RAG e historial de resultados.

Su finalidad es facilitar la comprensión de los factores considerados por el modelo y de sus resultados, sin constituir una herramienta de diagnóstico médico.

7.2.3. Sistema externo

Google Gemini API: proveedor externo de servicios de inteligencia artificial utilizados para generar representaciones vectoriales durante la ingesta documental y para apoyar la generación de contenido educativo mediante el sistema RAG.

Los servicios de n8n y Supabase se consideran parte de la solución PULSO desde el punto de vista lógico, aunque se encuentren alojados fuera del stack local de Docker Compose.

Por esta razón, sus responsabilidades internas se detallan en el diagrama de contenedores del nivel 2.

7.2.4. Relaciones

Origen	Destino	Interacción
Persona usuaria	PULSO	Se registra, inicia sesión, consulta predicciones, simula escenarios y revisa historial y recomendaciones
Responsable técnico	PULSO	Administra, verifica y mantiene los servicios y recursos documentales
PULSO	Google Gemini API	Utiliza servicios externos de IA para procesamiento documental y generación educativa mediante HTTPS/API

7.2.5. Diagrama de contexto

C4 · Nivel 1 — Contexto

PULSO — Diagrama de contexto (C4, nivel 1)

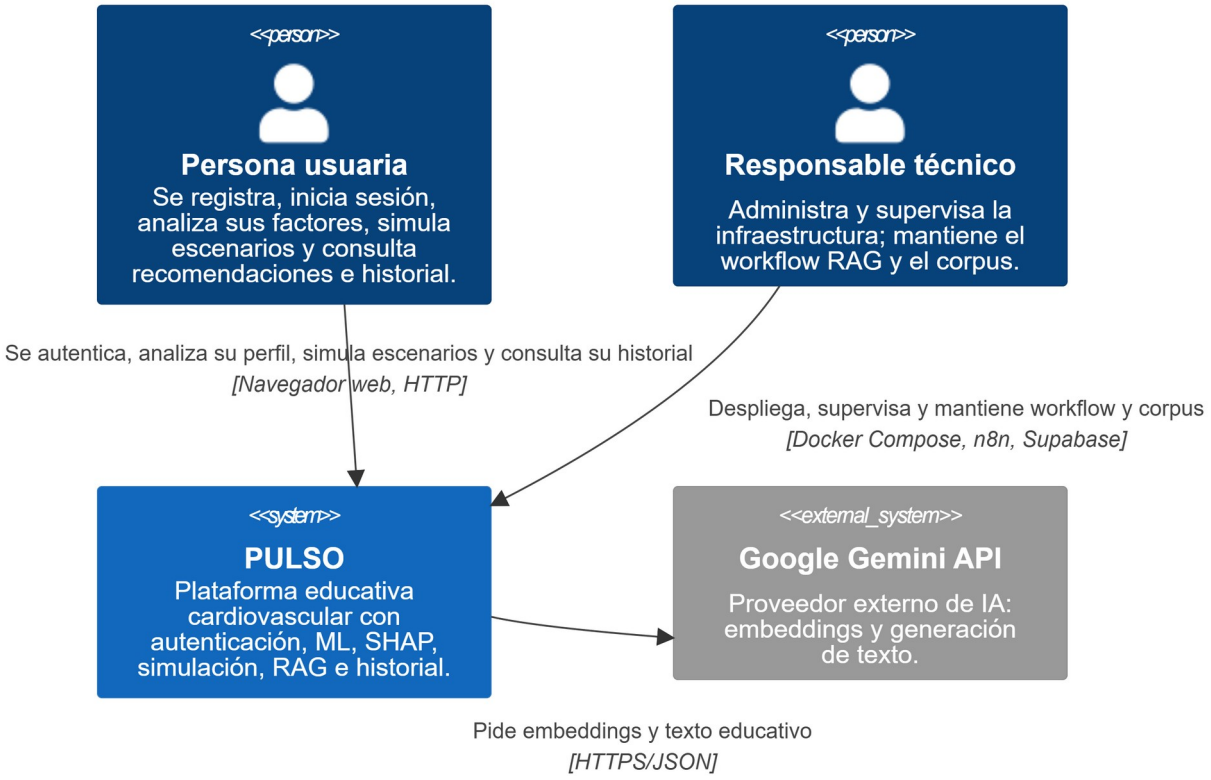


Figura 7.1 · Contexto de PULSO.

El diagrama permite identificar el propósito general de la plataforma, sus actores y su dependencia externa principal sin introducir detalles internos de implementación.

7.3. C4 Nivel 2 — Contenedores del sistema

El segundo nivel del modelo C4 descompone PULSO en sus principales aplicaciones, servicios y almacenes de datos, mostrando sus tecnologías, responsabilidades y relaciones.

En el modelo C4, el término *contenedor* se refiere a una unidad de ejecución o almacenamiento, y no necesariamente a un contenedor Docker.

7.3.1. Proxy inverso NGINX

NGINX funciona como punto de entrada a la plataforma.

Recibe las solicitudes provenientes del navegador y las dirige hacia el servidor del frontend o hacia el backend, según la ruta solicitada.

En la configuración local, el servicio expone el puerto 80 y permite centralizar el acceso a la aplicación.

7.3.2. Frontend web

El frontend está desarrollado con React, TypeScript y Vite.

Es responsable de:

- Presentar las pantallas de registro e inicio de sesión.
- Consultar el historial de resultados asociado a la cuenta.
- Presentar el formulario de evaluación cardiovascular.
- Capturar las variables requeridas por el modelo.
- Mostrar la probabilidad y clasificación obtenidas.
- Presentar los principales factores explicativos mediante SHAP.
- Permitir modificaciones de variables para simular escenarios.
- Mostrar recomendaciones educativas y referencias documentales.

La interfaz utiliza una distribución adaptable a diferentes anchos de pantalla. La validación funcional completa en dispositivos móviles constituye una comprobación independiente de esta característica visual.

7.3.3. Backend API

El backend está desarrollado en Python mediante FastAPI y expone una API REST.

Su responsabilidad principal consiste en coordinar las funcionalidades de la plataforma.

Realiza las siguientes operaciones:

- Registro, autenticación JWT y protección de endpoints.
- Persistencia de cuentas e historial resumido en PostgreSQL.
- Consulta paginada del historial de análisis.
- Validación de solicitudes.
- Construcción de objetos del dominio.
- Ejecución de casos de uso.
- Comunicación con ML Service.
- Almacenamiento temporal del contexto de análisis.
- Solicitud de recomendaciones al workflow RAG.
- Coordinación de simulaciones.
- Traducción de errores del dominio a respuestas HTTP.

El backend mantiene separadas las reglas de negocio de las implementaciones concretas de infraestructura.

7.3.4. Servicio de Machine Learning

ML Service es un servicio independiente desarrollado con Python y FastAPI.

Integra el modelo XGBoost de PULSO y la biblioteca SHAP.

Sus funciones principales son:

- Procesar los perfiles cardiovasculares.
- Generar probabilidades y clasificaciones.
- Obtener explicaciones individuales mediante SHAP.
- Ejecutar inferencias sobre escenarios hipotéticos.
- Proporcionar resultados estructurados al backend.

El backend se comunica con ML Service mediante HTTP/JSON.

7.3.5. Sistema RAG

La generación de recomendaciones educativas se implementa mediante workflows de n8n.

El sistema recibe información estructurada proveniente del backend, que incluye los datos del perfil cardiovascular, la predicción y los principales factores explicativos de SHAP.

El workflow coordina la recuperación documental y la generación de contenido educativo fundamentado.

Una decisión importante de la arquitectura es que **el RAG reutiliza la predicción y la explicación previamente calculadas**, sin realizar una nueva inferencia del modelo cardiovascular.

7.3.6. Base vectorial Supabase

El sistema RAG utiliza Supabase, basado en PostgreSQL y pgvector, para almacenar y recuperar información documental.

La configuración productiva utiliza la tabla:

public.documents_pulso_test

La recuperación semántica se realiza mediante la función:

public.match_documents_pulso_test

El corpus está compuesto por 15 documentos normalizados y 160 fragmentos vectorizados mediante models/gemini-embedding-001, con embeddings de 3072 dimensiones.

La denominación de la tabla conserva el sufijo _test debido a su origen como entorno de pruebas, aunque actualmente forma parte de la arquitectura operativa de producción.

Supabase se utiliza para persistencia documental y no constituye una base de datos de historiales personales de usuarios.

7.3.7. Base de datos relacional PostgreSQL

PostgreSQL 16 es un servicio del stack Docker Compose local. El backend accede mediante SQLAlchemy y psycopg para registrar cuentas, verificar identidades y conservar los resultados resumidos de cada análisis en las tablas users y analyses.

Se trata de un almacenamiento diferente de Supabase, que contiene exclusivamente el corpus documental del RAG.

7.3.8. Almacenamiento temporal

El backend utiliza InMemoryAnalysisRepository como implementación del puerto AnalysisRepository.

Este mecanismo conserva temporalmente los contextos necesarios para continuar los flujos de análisis y recomendaciones educativas.

Su configuración establece:

- Tiempo de vida de los registros: 900 segundos.
- Capacidad máxima: 100 entradas.
- Almacenamiento local en memoria del proceso.

Este repositorio no es persistente ni compartido entre múltiples procesos del backend.

Por ello, se representa como una responsabilidad interna del backend y no como un contenedor independiente de base de datos.

7.3.9. Comunicación entre contenedores

Origen	Destino	Protocolo	Propósito
Navegador	NGINX	HTTP en entorno local	Acceso a la aplicación
NGINX	Frontend	HTTP	Entrega de archivos HTML, JavaScript y CSS
NGINX	Backend	HTTP/JSON	Enrutamiento de solicitudes /api/*
Backend	PostgreSQL	SQL mediante SQLAlchemy/psycopg	Registro, consulta de usuarios y persistencia del historial
Backend	ML Service	HTTP/JSON	Predicción, explicación SHAP y simulación
Backend	n8n	HTTP/JSON	Solicitud de recomendaciones educativas
n8n	Supabase	HTTPS/API	Recuperación documental mediante similitud vectorial
n8n	Google Gemini API	HTTPS/JSON	Operaciones de IA utilizadas por el workflow

La aplicación React se ejecuta en el navegador una vez cargados sus recursos.

Las solicitudes a la API se realizan a través de NGINX, que funciona como proxy inverso.

Los servicios NGINX, frontend, backend, ML Service y PostgreSQL forman parte del stack local definido mediante Docker Compose.

n8n y Supabase se consideran componentes lógicos de PULSO, aunque su infraestructura de ejecución se encuentre fuera de ese stack.

7.3.10. Diagrama de contenedores

C4 · Nivel 2 — Contenedores

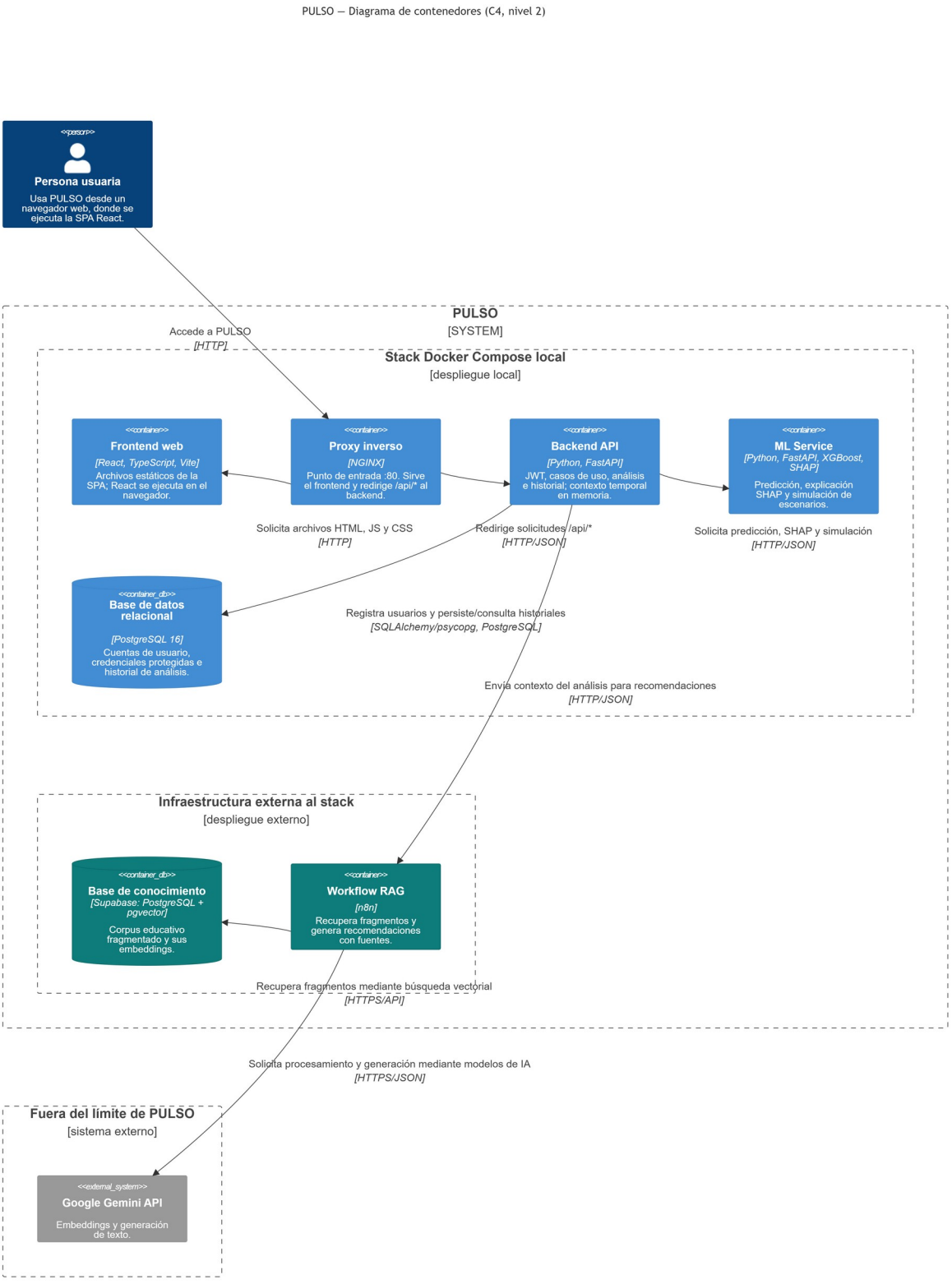


Figura 7.2 · Contenedores e integraciones de PULSO.

El diagrama permite comprender cómo se distribuyen las responsabilidades entre las aplicaciones y servicios que componen la plataforma.

7.4. C4 Nivel 3 — Componentes del backend

El tercer nivel del modelo C4 representa la estructura interna del contenedor Backend API.

El backend de PULSO implementa una adaptación de Clean Architecture basada en la separación de responsabilidades y la inversión de dependencias.

Su organización se divide en cuatro capas:

1. Dominio.
2. Aplicación.
3. Infraestructura.
4. Interfaces HTTP.

Las dependencias del código deben respetar la independencia del dominio respecto de las tecnologías externas.

7.4.1. Capa de dominio

Ubicación: backend/app/domain/

Contiene los objetos centrales, las reglas de negocio, las excepciones y los contratos que utilizan los casos de uso.

Su estructura principal es:

Archivo	Responsabilidad
entities.py	Define los objetos de dominio como CardiovascularProfile, Prediction, Explanation, AnalysisContext, RecommendationReport y SimulationOutcome
rules.py	Contiene reglas de coherencia del perfil y del IMC
errors.py	Define excepciones propias del dominio
ports.py	Establece los contratos que deben cumplir los servicios externos y repositorios

Junto con los objetos cardiovasculares, el dominio contempla AuthUser y AnalysisHistoryEntry, que representan la identidad y los resultados históricos asociados a cuentas.

Los puertos principales son:

CardiovascularModel: abstrae las operaciones de predicción, explicabilidad y simulación.

RecommendationGenerator: abstrae la generación de recomendaciones educativas.

AnalysisRepository: abstrae el almacenamiento y la recuperación temporal de contextos de análisis.

Otros puertos gestionan repositorios de usuarios e historial, hashing de contraseñas y emisión de tokens, incluidos PasswordHasher y AccessTokenIssuer.

El dominio utiliza estructuras de Python y no depende de FastAPI, Pydantic, httpx ni de implementaciones concretas de infraestructura.

7.4.2. Capa de aplicación

Ubicación: backend/app/application/use_cases/

Contiene los casos de uso responsables de coordinar los procesos principales.

AnalyzeProfile

Coordina el análisis cardiovascular.

Recibe el perfil, solicita una predicción y una explicación mediante el puerto CardiovascularModel y conserva el contexto obtenido a través de AnalysisRepository.

Devuelve los resultados y el identificador temporal asociado al análisis.

RequestRecommendations

Coordina la generación de información educativa.

Recupera el contexto previamente almacenado mediante AnalysisRepository y solicita las recomendaciones utilizando RecommendationGenerator.

Este caso de uso evita volver a ejecutar el modelo predictivo únicamente para obtener información educativa.

SimulateScenario

Coordina la simulación de escenarios.

Procesa las modificaciones permitidas y utiliza CardiovascularModel para obtener los resultados comparativos del escenario original y el modificado.

RegisterUser: crea una cuenta con correo único y contraseña protegida.

AuthenticateUser: comprueba credenciales y permite emitir el token JWT de acceso.

ListAnalysisHistory: consulta el historial paginado del usuario autenticado.

Los casos de uso dependen de contratos definidos en el dominio, sin conocer los detalles concretos de comunicación con ML Service, n8n, SQLAlchemy o las bibliotecas de seguridad.

7.4.3. Capa de infraestructura

Ubicación: backend/app/infrastructure/

Contiene las implementaciones concretas de los contratos del dominio.

Componente	Puerto implementado	Responsabilidad
MLServiceClient	CardiovascularModel	Comunicación HTTP con ML Service
N8nRecommendationClient	RecommendationGenerator	Comunicación HTTP con el workflow RAG
InMemoryAnalysisRepository	AnalysisRepository	Almacenamiento temporal de análisis
SQLAlchemyUserRepository	Repositorio de usuarios	Persistencia de cuentas en PostgreSQL
SQLAlchemyAnalysisHistoryRepository	Repositorio de historial	Guarda y recupera resultados por usuario
Argon2PasswordHasher	PasswordHasher	Hashing y verificación de contraseñas
JWTAccessTokens	AccessTokenIssuer	Emisión y validación de tokens JWT
Settings	No aplica	Configuración del sistema mediante variables de entorno

Los clientes externos utilizan httpx para comunicarse mediante HTTP.

La infraestructura depende de los contratos del dominio, mientras que el dominio permanece independiente de sus implementaciones concretas.

7.4.4. Capa de interfaces HTTP

Ubicación: backend/app/interfaces/http/

Esta capa utiliza FastAPI y Pydantic para exponer las operaciones del sistema mediante una API REST.

Su estructura comprende:

Archivo	Responsabilidad
routes/analysis.py	Endpoints /api/analyze, /api/rag, /api/simulate y consulta del historial, según la separación de rutas implementada
routes/auth.py	Registro, inicio de sesión y consulta de identidad (/api/auth/*)
routes/health.py	Endpoint /api/health
schemas.py	Validación y serialización de contratos JSON
dependencies.py	Composición e inyección de dependencias
errors.py	Conversión de errores del dominio a respuestas HTTP

La dependencia get_current_user valida credenciales HTTP Bearer y determina la identidad en las operaciones protegidas.

El archivo dependencies.py constituye el punto de composición de la arquitectura.

Su responsabilidad es construir los casos de uso, proporcionar las implementaciones concretas de los puertos y relacionar los componentes necesarios para procesar las solicitudes.

7.4.5. Relaciones y regla de dependencias

La arquitectura utiliza inversión de dependencias para evitar que los casos de uso dependan directamente de los servicios externos.

Las relaciones principales son:

Origen	Destino	Relación
Rutas FastAPI	Casos de uso	Ejecutan operaciones de aplicación
AnalyzeProfile	CardiovascularModel	Utiliza contrato de predicción
AnalyzeProfile	AnalysisRepository	Utiliza contrato de almacenamiento
RequestRecommendations	AnalysisRepository	Recupera contexto
RequestRecommendations	RecommendationGenerator	Solicita recomendaciones
SimulateScenario	CardiovascularModel	Solicita simulación
MLServiceClient	CardiovascularModel	Implementa contrato
N8nRecommendationClient	RecommendationGenerator	Implementa contrato
InMemoryAnalysisRepository	AnalysisRepository	Implementa contrato
RegisterUser / AuthenticateUser	Puertos de seguridad y usuarios	Gestionan registro y autenticación
ListAnalysisHistory	Repositorio de historial	Consulta resultados según la identidad
Repositorios SQLAlchemy	PostgreSQL	Persisten cuentas y resultados
dependencies.py	Casos de uso y adaptadores	Construye y conecta dependencias

Debe diferenciarse entre una dependencia de código, la implementación de un contrato y una comunicación de ejecución.

Por ejemplo, MLServiceClient implementa CardiovascularModel, mientras que en ejecución realiza una solicitud HTTP hacia el servicio de Machine Learning.

7.4.6. Verificación automatizada de dependencias

El backend dispone de una prueba automatizada de arquitectura ubicada en:

backend/tests/test_architecture.py

Esta prueba utiliza el módulo ast de Python para analizar instrucciones de importación y detectar dependencias prohibidas entre las capas.

Las restricciones contemplan:

- El dominio no debe depender de aplicación, infraestructura, interfaces HTTP ni frameworks externos utilizados en esas capas.
- La aplicación debe depender de los contratos y objetos del dominio, sin acoplarse directamente a los adaptadores de infraestructura.
- La infraestructura no debe depender de la capa HTTP.
- La capa de interfaces puede realizar la composición de los componentes concretos.

Este mecanismo contribuye a verificar que la organización del código conserve las reglas de dependencia establecidas.

La comprobación estática de imports complementa las pruebas unitarias y de integración, pero no garantiza por sí sola la ausencia de todas las formas posibles de acoplamiento.

7.4.7. Diagrama de componentes

C4 · Nivel 3 — Componentes del backend

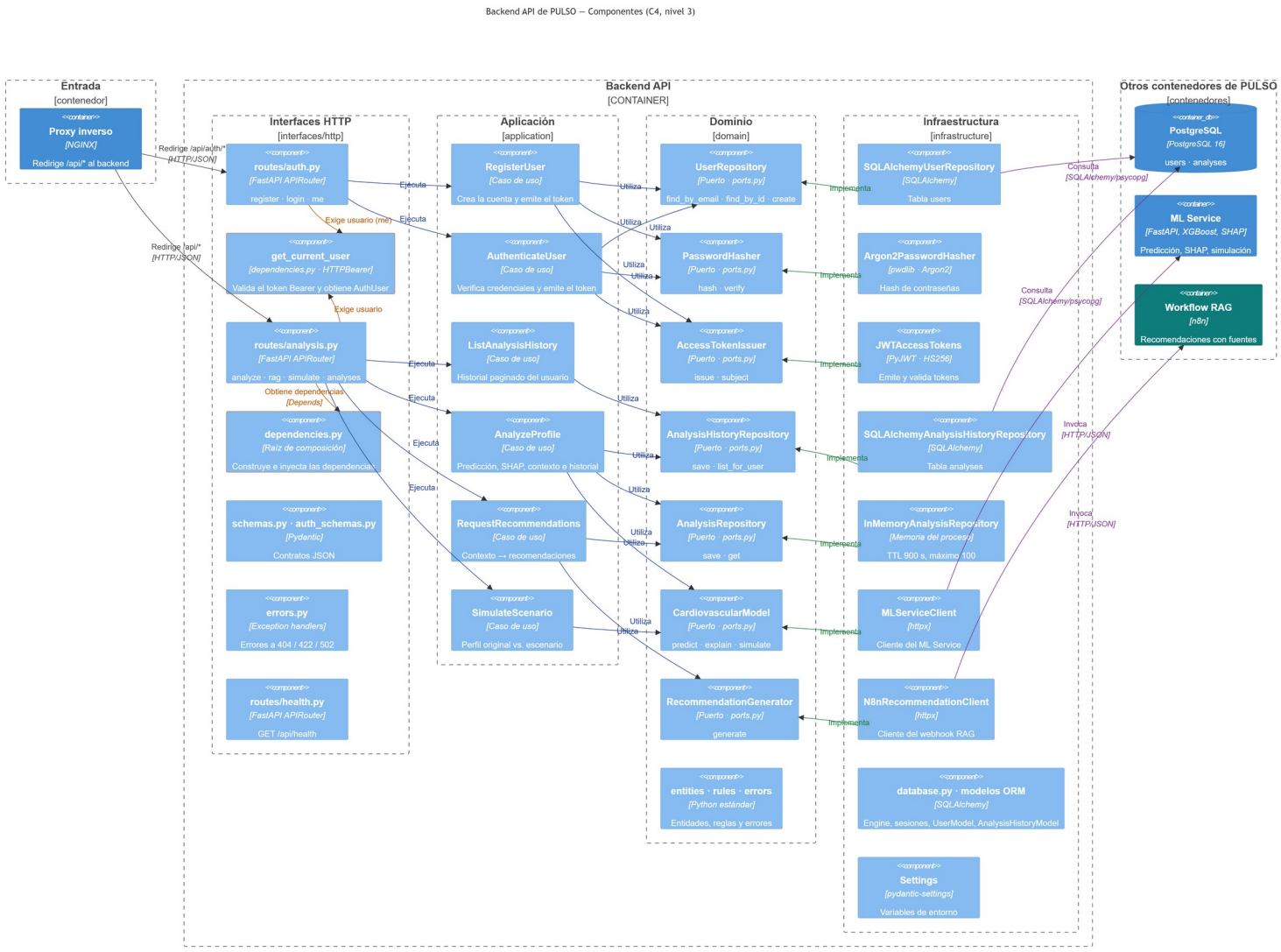


Figura 7.3 · Componentes del backend FastAPI.

El diagrama permite identificar los componentes centrales del backend y comprobar que las responsabilidades se encuentran distribuidas entre capas diferenciadas.

7.5. Flujo de análisis cardiovascular

El flujo de análisis corresponde al endpoint POST /api/analyze.

Su ejecución comprende:

5. El usuario introduce sus factores cardiovasculares mediante el frontend.
6. La solicitud llega al backend a través de NGINX.
7. FastAPI valida el contrato JSON mediante Pydantic y las reglas correspondientes del dominio.
8. La ruta construye un objeto CardiovascularProfile.
9. El caso de uso AnalyzeProfile solicita la predicción y explicación al puerto CardiovascularModel.
10. MLServiceClient se comunica con ML Service para ejecutar la inferencia y obtener la explicación SHAP.
11. El contexto del análisis se conserva temporalmente mediante AnalysisRepository.
12. El backend devuelve el identificador del análisis, la predicción y los factores explicativos.
13. El frontend presenta los resultados al usuario.

Este flujo separa la coordinación del análisis de la ejecución del modelo predictivo.

7.6. Flujo de recomendaciones educativas

El endpoint POST /api/rag permite solicitar recomendaciones educativas utilizando un análisis previamente calculado.

El proceso comprende:

14. El frontend envía el identificador temporal del análisis.
15. El backend recupera el contexto mediante AnalysisRepository.
16. RequestRecommendations utiliza RecommendationGenerator para solicitar información educativa.
17. N8nRecommendationClient invoca el webhook correspondiente de n8n.
18. El workflow RAG consulta el corpus documental vectorizado en Supabase.
19. El sistema utiliza los servicios de IA configurados para generar recomendaciones educativas fundamentadas.
20. n8n devuelve una respuesta estructurada con información educativa y referencias documentales.
21. El backend entrega la respuesta al frontend.

El proceso no requiere ejecutar nuevamente ML Service para generar recomendaciones.

7.7. Flujo de simulación de escenarios

El endpoint POST /api/simulate permite evaluar escenarios hipotéticos a partir de modificaciones de determinadas variables.

La operación comprende:

22. El usuario selecciona las variables que desea modificar.
23. El frontend envía el escenario al backend.
24. FastAPI valida los datos recibidos.
25. El caso de uso SimulateScenario solicita la simulación mediante CardiovascularModel.
26. ML Service ejecuta el modelo considerando el escenario modificado y la coherencia de las variables derivadas.
27. Se obtiene el resultado comparativo entre el escenario de referencia y el escenario simulado.

28. El backend devuelve los resultados al frontend.

Las diferencias obtenidas representan cambios en la salida del clasificador y no estimaciones causales de intervenciones clínicas.

7.8. Gestión de usuarios e historial implementados

La arquitectura incorpora un módulo de registro e inicio de sesión mediante JWT, contraseñas protegidas con Argon2, operaciones autenticadas y un historial persistente de análisis por cuenta.

En el dominio se identifican las estructuras AuthUser y AnalysisHistoryEntry y los contratos relativos a usuarios, historial y seguridad. En aplicación se utilizan RegisterUser, AuthenticateUser y ListAnalysisHistory.

La infraestructura proporciona repositorios SQLAlchemy para PostgreSQL y adaptadores de hashing y tokens. Las interfaces HTTP exponen /api/auth/register, /api/auth/login, /api/auth/me y /api/analyses, y utilizan get_current_user para comprobar la identidad.

El historial persiste únicamente resultados seleccionados; el perfil cardiovascular completo y SHAP continúan disponibles temporalmente en memoria durante el flujo educativo.

No se presentan como implementados la administración de múltiples roles, los refresh tokens, el consentimiento registrado ni la eliminación de cuentas. Estas capacidades requerirían ampliaciones diferenciadas del diseño.

7.9. Decisiones técnicas y justificación

Decisión técnica	Justificación
React y TypeScript	Construcción de una interfaz web basada en componentes
FastAPI	Implementación de APIs y coordinación de operaciones en Python
Clean Architecture adaptada	Separación de responsabilidades e inversión de dependencias
ML Service independiente	Aislamiento de la inferencia predictiva
XGBoost	Implementación del modelo supervisado de clasificación
SHAP	Explicación local de las predicciones
n8n	Orquestación del sistema RAG
PostgreSQL y SQLAlchemy	Persistencia de cuentas e historial con esquema versionado
JWT y Argon2	Autenticación de acceso y protección de credenciales
Supabase y pgvector	Persistencia y recuperación semántica documental
Google Gemini API	Generación de embeddings y contenido mediante servicios de IA
Repositorio temporal en memoria	Reutilización de resultados sin historial persistente
NGINX	Centralización del acceso a frontend y backend
Docker Compose	Ejecución coordinada y reproducible del stack principal
Pruebas de arquitectura	Verificación automatizada de las reglas de dependencias

7.10. Limitaciones arquitectónicas

La arquitectura implementada presenta restricciones que deben considerarse durante su operación y evolución.

Almacenamiento temporal: el repositorio de análisis conserva información en memoria del proceso, por lo que no proporciona persistencia permanente ni almacenamiento compartido entre múltiples instancias.

Dependencias externas: el funcionamiento del sistema RAG depende de la disponibilidad de n8n, Supabase y los servicios de inteligencia artificial configurados.

Disponibilidad: la ejecución satisfactoria de los contenedores locales no garantiza automáticamente la disponibilidad de todas las integraciones externas.

Seguridad del despliegue: los mecanismos de comunicación y exposición pública deben configurarse de acuerdo con las condiciones del entorno de ejecución.

Validación clínica: los resultados del modelo y las recomendaciones educativas no constituyen diagnósticos médicos ni sustituyen una evaluación clínica.

Gestión de usuarios: están implementados el registro, el inicio de sesión y el historial; la gestión de roles y consentimiento permanece fuera de las capacidades acreditadas de la versión documentada.

7.11. Conclusión

La arquitectura de PULSO se fundamenta en una separación modular de responsabilidades entre la interfaz web, el backend, PostgreSQL para cuentas e historial, el servicio de Machine Learning y el sistema RAG.

El backend implementa una adaptación de Clean Architecture que utiliza entidades de dominio, casos de uso, puertos y adaptadores para reducir el acoplamiento entre la lógica principal y los servicios externos.

Los tres niveles del modelo C4 permiten representar la solución desde perspectivas complementarias:

- **Nivel 1:** contexto general, actores y dependencias externas.
- **Nivel 2:** aplicaciones, servicios, tecnologías y comunicaciones.
- **Nivel 3:** componentes internos del backend y reglas de dependencia.

Esta organización facilita la comprensión de la arquitectura existente, la identificación de responsabilidades y la evolución del sistema, manteniendo separadas las funcionalidades implementadas de aquellas que permanecen como propuestas futuras.